

Assignment 5: Web-based Vulnerabilities & Attacks

Author: Δημήτρης Σταθούλιας

AM: 2018030109

Date: 9/12/2025

Login Page SQL Injection

- **Explanation:** This vulnerability allows an attacker to input an SQL query in the password field when attempting to login, which the server accepts as a legitimate query and can be exploited. For this approach, I initially attempted to inject a query that always returns true (' OR '1' = '1'), which failed, but after inspecting the source code I noticed there were filters in place before passing the password to the database. To bypass the filters, I simply removed the spaces in the query, and the database granted me access.
- **URL:** <http://139.91.71.5:11337>
- **Form Field:** password
- **Payload:** 'or'1'='1

DOM-XSS Vulnerability

- **Explanation:** This vulnerability allows an attacker to add scripts to the URL of the page that the source code writes to the DOM. To exploit that, I attempted to add a script to the URL of a page, where it reads the URL fragment and writes in to the page, that simply raises an alert 'DOM-XSS' when the page is (re)loaded. At that point, the page displayed an error message which, after I inspected the source code, was due to the CSP

security measure set in place. To bypass that, I needed to add the hardcoded nonce clause (nonce="1337x0xyz") to my script injection and the script was successfully executed.

- **URL:** <http://139.91.71.5:11337/dashboard>
- **URL Fragment:** uname (after the #)
- **Payload:** `<script nonce="1337x0xyz">alert("DOM-XSS")</script>`
- **Malicious URL:** [http://139.91.71.5:11337/dashboard#uname<script nonce="1337x0xyz">alert\("DOM-XSS"\)</script>](http://139.91.71.5:11337/dashboard#uname<script nonce=)

Reflected XSS Vulnerability

- **Explanation:** This vulnerability reflects user input onto the results page without proper escaping. To exploit this vulnerability, I injected a script into the search bar of the page that, similarly to the DOM-XSS exploit, raises an alert 'Reflected-XSS' when the page is (re)loaded. Following the DOM-XSS approach, I added the nonce clause to the script for it to bypass the CSP and be properly executed.
- **URL:** <http://139.91.71.5:11337/dashboard>
- **Form Field:** Item search bar
- **Payload:** `<script nonce="1337x0xyz">alert("Reflected-XSS")</script>`

Search Bar SQL Injection

- **Explanation:** This vulnerability is the same as the password SQL injection performed earlier. This time the goal is to inject an SQL query into the item search bar to display the users table contents. When inspecting the source code for the previous SQL exploit, I noticed there was a rule in place that prevented me from injecting multiple queries as the character ";" was to be rejected. To bypass that, I used the UNION operator to combine results from the items and users tables and display

the columns username, password instead. Doing that, I was able to retrieve the admin password “\$#youCantCrackMyPassword#\$”.

- **URL:** <http://139.91.71.5:11337/dashboard>
- **Form Field:** Item search bar
- **Payload:** ' UNION SELECT username, password, 1 FROM users - -

Note that the “- -” characters are required to comment out any leftover code after the injection point to prevent any errors from occurring.

Open Redirect Vulnerability

- **Explanation:** This vulnerability allows an attacker to specify a redirect URL that the site performs without validating its security. When inspecting the source code, I noticed the /go endpoint accepts a user-specified destination via the “to” parameter. To exploit that, I used that endpoint to force an automatic redirect to the specified external <http://example.com> website.
- **URL:** <http://139.91.71.5:11337/go>
- **Parameter:** to
- **Payload:** <http://example.com/>
- **Malicious URL:** <http://139.91.71.5:11337/go?to=http://example.com/>

LFI Vulnerability

- **Explanation:** This vulnerability allows an attacker to access non-intended files inside or even outside the app directory, which is possible because the application accepts user specified “show” parameters and passes it directly to the file system without validating it. To fulfill the

assignment's requirements and find the secret flag in the remote application I first used the admin password found in the search bar SQL injection exploit to gain access to the admin page and its view files function. Then, I used the LFI exploit to search the files directory for any files that could contain the secret flag requested via a trial-and-error method. The potential file names attempted were: flag.txt, supersecretflag.txt, secretflag.txt, theflag.txt, realflag.txt. Finally, the flag was found in the files/realflag.txt file and is displayed below.

- **URL:** <http://139.91.71.5:11337/admin>
- **Parameter:** show
- **Payload:** /files/realflag
- **Malicious URL:** <http://139.91.71.5:11337/admin?show=/files/realflag>
- **Flag:**
TUC{972f02eb8227012f0b9954e95efc4001a28290ef48047a922efc2a4db40954e6}